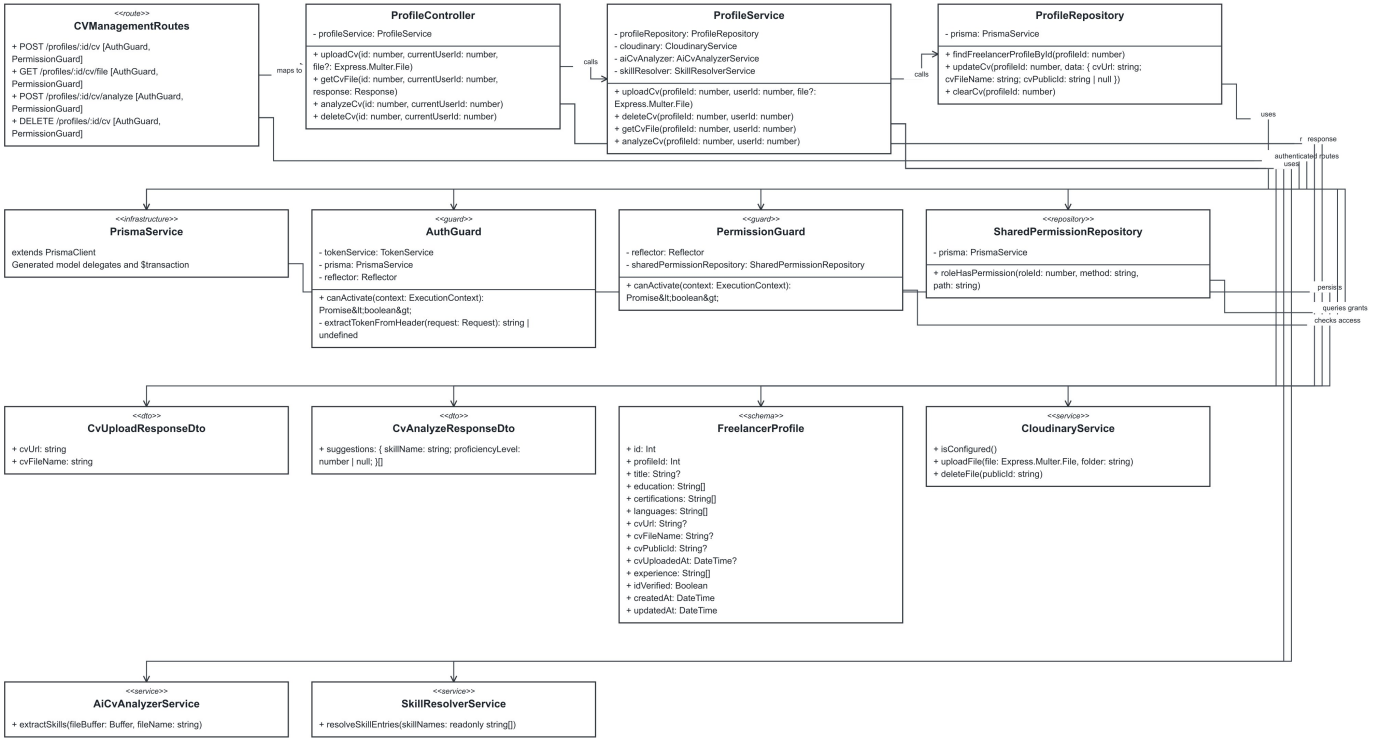

<<route>>
CVManagementRoutes
+ POST /profiles/:id/cv [AuthGuard, PermissionGuard]
+ GET /profiles/:id/cv/file [AuthGuard, PermissionGuard]
+ POST /profiles/:id/cv/analyze [AuthGuard, PermissionGuard]
+ DELETE /profiles/:id/cv [AuthGuard, PermissionGuard]
maps to
ProfileController
- profileService: ProfileService
+ uploadCv(id: number, currentUserId: number, file?: Express.Multer.File)
+ getCvFile(id: number, currentUserId: number, response: Response)
+ analyzeCv(id: number, currentUserId: number)
+ deleteCv(id: number, currentUserId: number)
calls
ProfileService
- profileRepository: ProfileRepository
- cloudinary: CloudinaryService
- aiCvAnalyzer: AiCvAnalyzerService
- skillResolver: SkillResolverService
+ uploadCv(profileId: number, userId: number, file?: Express.Multer.File)
+ deleteCv(profileId: number, userId: number)
+ getCvFile(profileId: number, userId: number)
+ analyzeCv(profileId: number, userId: number)
calls
ProfileRepository
- prisma: PrismaService
+ findFreelancerProfileById(profileId: number)
+ updateCv(profileId: number, data: { cvUrl: string; cvFileName: string; cvPublicId: string | null })
+ clearCv(profileId: number)
uses
r
response
t
authenticated routes
uses
<<infrastructure>>
PrismaService
extends PrismaClient
Generated model delegates and $transaction
<<guard>>
AuthGuard
- tokenService: TokenService
- prisma: PrismaService
- reflector: Reflector
+ canActivate(context: ExecutionContext): Promise<boolean>
- extractTokenFromHeader(request: Request): string | undefined
<<guard>>
PermissionGuard
- reflector: Reflector
- sharedPermissionRepository: SharedPermissionRepository
+ canActivate(context: ExecutionContext): Promise<boolean>
<<repository>>
SharedPermissionRepository
- prisma: PrismaService
+ roleHasPermission(roleId: number, method: string, path: string)
persists
queries grants
checks access
<<dto>>
CvUploadResponseDto
+ cvUrl: string
+ cvFileName: string
<<dto>>
CvAnalyzeResponseDto
+ suggestions: { skillName: string; proficiencyLevel: number | null; }[]
<<schema>>
FreelancerProfile
+ id: Int
+ profileId: Int
+ title: String?
+ education: String[]
+ certifications: String[]
+ languages: String[]
+ cvUrl: String?
+ cvFileName: String?
+ cvPublicId: String?
+ cvUploadedAt: DateTime?
+ experience: String[]
+ idVerified: Boolean
+ createdAt: DateTime
+ updatedAt: DateTime
<<service>>
CloudinaryService
+ isConfigured()
+ uploadFile(file: Express.Multer.File, folder: string)
+ deleteFile(publicId: string)
<<service>>
AiCvAnalyzerService
+ extractSkills(fileBuffer: Buffer, fileName: string)
<<service>>
SkillResolverService
+ resolveSkillEntries(skillNames: readonly string[])